

1. Problema 1

Maximizar la función $h(\beta, \beta_0)$ definida como:

$$h(\beta, \beta_0) = \sum_{j=1}^n [y_j \log \pi_j + (1 - y_j) \log(1 - \pi_j)] \quad (1.1)$$

donde

$$\pi_j = \pi_j(\beta, \beta_0) = \frac{1}{1 + \exp(-x_j^T \beta - \beta_0)}. \quad (1.2)$$

Usar el algoritmo de descenso del gradiente para maximizar $h(\beta, \beta_0)$ con respecto a β y β_0 . Comparar los resultados utilizando un tamaño de paso fijo y un tamaño de paso inexacto con backtracking. Graficar el número de iteraciones k vs $\|g_k\|$ y el número de iteraciones k vs $-h_k$.

1.1. Método

Para maximizar la verosimilitud, es necesario introducir un signo negativo en la función objetivo, de modo que los algoritmos de optimización puedan maximizarla. A continuación, se presenta el cálculo del gradiente de la función $h(\beta, \beta_0)$.

1.1.1. Cálculo del gradiente

El gradiente de $h(\beta, \beta_0)$ se calcula como:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n [y_j \nabla \log \pi_j + (1 - y_j) \nabla \log(1 - \pi_j)]. \quad (1.3)$$

Desarrollando esta expresión, se obtiene:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n \left[y_j \frac{1}{\pi_j} \nabla \pi_j + (1 - y_j) \frac{1}{1 - \pi_j} (-\nabla \pi_j) \right]. \quad (1.4)$$

Simplificando, se llega a:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n \left[\frac{y_j}{\pi_j} - \frac{1 - y_j}{1 - \pi_j} \right] \nabla \pi_j. \quad (1.5)$$

Factorizando, se obtiene:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n \left[\frac{y_j(1 - \pi_j) - (1 - y_j)\pi_j}{\pi_j(1 - \pi_j)} \right] \nabla \pi_j. \quad (1.6)$$

Simplificando el numerador:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n \left[\frac{y_j - \pi_j y_j - \pi_j + y_j \pi_j}{\pi_j (1 - \pi_j)} \right] \nabla \pi_j. \quad (1.7)$$

Finalmente, se reduce a:

$$\nabla h(\beta, \beta_0) = - \sum_{j=1}^n \left[\frac{y_j - \pi_j}{\pi_j (1 - \pi_j)} \right] \nabla \pi_j. \quad (1.8)$$

1.1.2. Cálculo de $\nabla \pi_j$

El gradiente de π_j con respecto a β_0 es:

$$\begin{aligned} \nabla \pi_j &= - \frac{-\exp(-x_j^T \beta - \beta_0))}{(1 + \exp(-x_j^T \beta - \beta_0))^2} \\ &= \pi_j^2 \exp(-x_j^T \beta - \beta_0)) \\ &= \pi_j^2 (1 + \exp(-x_j^T \beta - \beta_0)) - \pi_j^2 \\ &= \pi_j^2 (\pi_j^{-1}) - \pi_j^2 \\ &= \pi_j^1 - \pi_j^2 \\ &= \pi_j (1 - \pi_j) \\ \nabla \pi_j &= \pi_j (1 - \pi_j). \end{aligned} \quad (1.9)$$

Para β_i con $i > 0$, el gradiente es:

$$\begin{aligned} \nabla \pi_j &= - \frac{-x_{ji} \exp(-x_j^T \beta - \beta_0))}{(1 + \exp(-x_j^T \beta - \beta_0))^2} \\ &= x_{ji} \pi_j^2 \exp(-x_j^T \beta - \beta_0)) \\ &= x_{ji} (\pi_j^2 (1 + \exp(-x_j^T \beta - \beta_0)) - \pi_j^2) \\ &= x_{ji} (\pi_j^2 (\pi_j^{-1}) - \pi_j^2) \\ &= x_{ji} (\pi_j^1 - \pi_j^2) \\ &= x_{ji} \pi_j (1 - \pi_j) \\ \nabla \pi_j &= x_{ji} \pi_j (1 - \pi_j). \end{aligned} \quad (1.10)$$

Sustituyendo estas expresiones en la ecuación del gradiente, se obtiene:

$$\frac{\partial h}{\partial \beta_0} = - \sum_{j=1}^n (y_j - \pi_j), \quad (1.11)$$

y para β_i con $i > 0$:

$$\frac{\partial h}{\partial \beta_i} = - \sum_{j=1}^n (y_j - \pi_j) x_{ji}. \quad (1.12)$$

1.2. Implementación

A continuación, se presenta un pseudocódigo sencillo para la implementación del descenso del gradiente con tamaño de paso fijo y backtracking.

1.2.1. Pseudocódigo para tamaño de paso fijo

Algorithm 1 Descenso del gradiente con tamaño de paso fijo

Require: Función objetivo $h(\beta, \beta_0)$, gradiente $\nabla h(\beta, \beta_0)$, punto inicial (β_0, β) , tamaño de paso α , tolerancia ϵ , número máximo de iteraciones N .

Ensure: Punto óptimo (β^*, β_0^*) .

```

1: Inicializar  $(\beta, \beta_0)$  con valores iniciales.
2: Calcular  $h_0 = h(\beta, \beta_0)$ .
3: Calcular  $g_0 = \nabla h(\beta, \beta_0)$ .
4: Inicializar  $k = 0$ .
5: while  $k < N$  y  $\|g_k\| > \epsilon$  do
6:   Actualizar  $(\beta, \beta_0)$ :  $(\beta_{k+1}, \beta_{0,k+1}) = (\beta_k, \beta_{0,k}) - \alpha g_k$ .
7:   Calcular  $h_{k+1} = h(\beta_{k+1}, \beta_{0,k+1})$ .
8:   Calcular  $g_{k+1} = \nabla h(\beta_{k+1}, \beta_{0,k+1})$ .
9:   Incrementar  $k = k + 1$ .
10: end while
11: return  $(\beta^*, \beta_0^*)$ .
```

1.2.2. Pseudocódigo para backtracking

Algorithm 2 Descenso del gradiente con backtracking

Require: Función objetivo $h(\beta, \beta_0)$, gradiente $\nabla h(\beta, \beta_0)$, punto inicial (β_0, β) , tamaño de paso inicial α , parámetros c_1, c_2 , tolerancia ϵ , número máximo de iteraciones N .

Ensure: Punto óptimo (β^*, β_0^*) .

```

1: Inicializar  $(\beta, \beta_0)$  con valores iniciales.
2: Calcular  $h_0 = h(\beta, \beta_0)$ .
3: Calcular  $g_0 = \nabla h(\beta, \beta_0)$ .
4: Inicializar  $k = 0$ .
5: while  $k < N$  y  $\|g_k\| > \epsilon$  do
6:   Aplicar backtracking para encontrar  $\alpha_k$  que satisfaga las condiciones de Wolfe.
7:   Actualizar  $(\beta, \beta_0)$ :  $(\beta_{k+1}, \beta_{0,k+1}) = (\beta_k, \beta_{0,k}) - \alpha_k g_k$ .
8:   Calcular  $h_{k+1} = h(\beta_{k+1}, \beta_{0,k+1})$ .
9:   Calcular  $g_{k+1} = \nabla h(\beta_{k+1}, \beta_{0,k+1})$ .
10:  Incrementar  $k = k + 1$ .
11: end while
12: return  $(\beta^*, \beta_0^*)$ .
```

1.3. Resultados

Los resultados incluyen gráficas del número de iteraciones k vs $\|g_k\|$ y el número de iteraciones k vs $-h_k$, que permiten visualizar la convergencia del algoritmo.

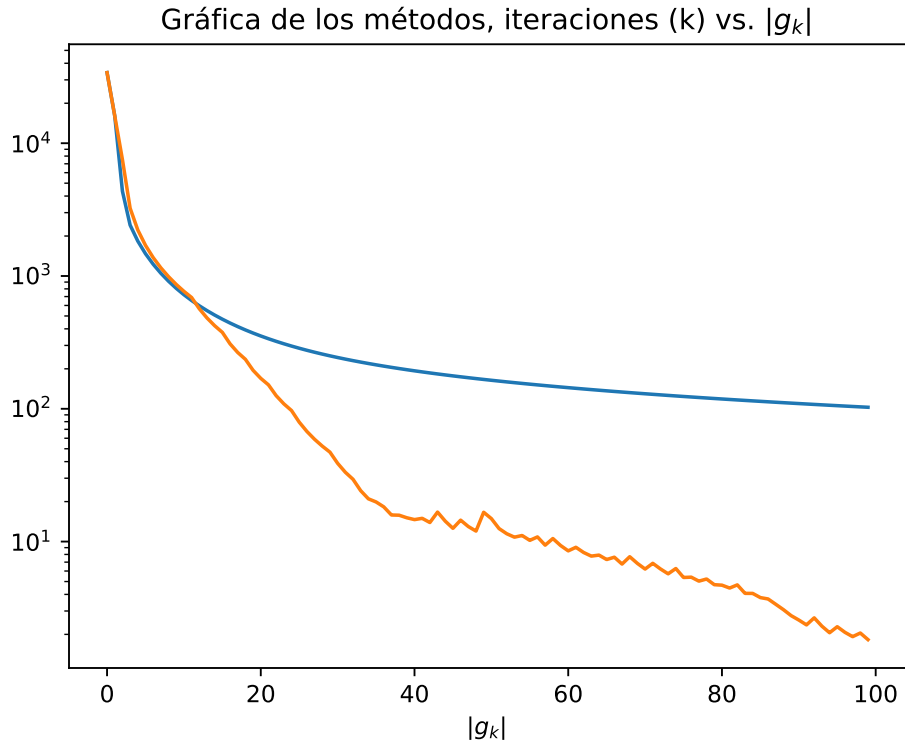


Figura 1.1: Gráfica de los métodos, iteraciones (k) vs. $\|g_k\|$

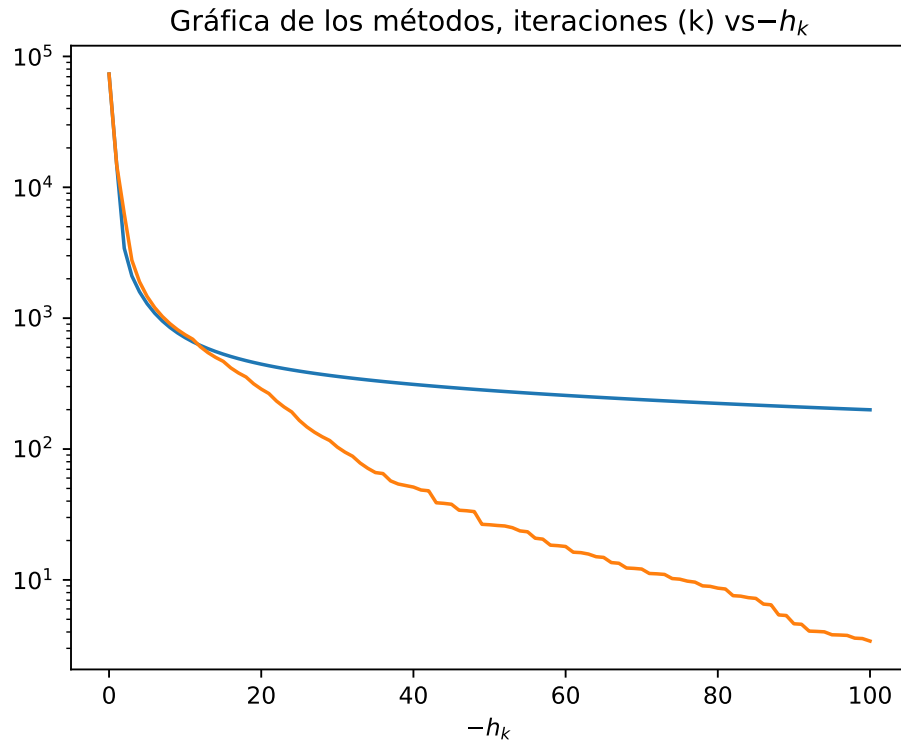


Figura 1.2: Gráfica de los métodos, iteraciones (k) vs $-h_k$

2. Problema 2

Utilizando los valores estimados $\hat{\beta}, \hat{\beta}_0$ del problema anterior, calcular el error:

$$\text{error} = \frac{1}{n} \sum_{i=1}^n \left| \mathbf{1}_{\pi_i(\hat{\beta}, \hat{\beta}_0) > 0.5}(x_i) - y_i \right| \quad (2.1)$$

donde los datos $(x_i, y_i)_{i=1}^n$ son tomados del conjunto de prueba.

2.1. Implementación

El cálculo del error se implementa en Python de la siguiente manera:

```
import numpy as np

def error(pi, y):
    indi = pi > 0.5
    indi = indi.astype(int)
    return np.mean(np.abs(indi - y))
```

2.2. Explicación del código

- La función `error` toma como entrada las probabilidades estimadas π y las etiquetas reales y .
- La variable `indi` es un arreglo de valores binarios (0 o 1) que indica si la probabilidad estimada es mayor que 0.5.
- La función `np.mean(np.abs(indi - y))` calcula el error promedio entre las predicciones y las etiquetas reales.

2.3. Resultados

El error calculado representa la proporción de clasificaciones incorrectas en el conjunto de prueba. Un valor cercano a 0 indica un buen ajuste del modelo, mientras que un valor cercano a 1 indica un mal ajuste.

A continuación, se presentan los errores de clasificación obtenidos para los métodos de tamaño de paso fijo y backtracking en los conjuntos de entrenamiento, validación y prueba:

Método	Train	Validation	Test
Simple	0.005372290292177191	0.00535279805352798	0.0028368794326241137
Backtracking	0.0	0.0009732360097323601	0.00047281323877068556

Cuadro 1: Errores de clasificación para los métodos de tamaño de paso fijo y backtracking.

2.4. Análisis de resultados

- El método de **tamaño de paso fijo** muestra errores bajos en todos los conjuntos de datos, con un error de entrenamiento de aproximadamente 0.0054, un error de validación de 0.0054 y un error de prueba de 0.0028.
- El método de **backtracking** logra un error de entrenamiento de 0.0, lo que indica un ajuste perfecto en el conjunto de entrenamiento. Además, obtiene errores de validación y prueba muy bajos (0.00097 y 0.00047, respectivamente), lo que sugiere una excelente generalización del modelo.
- En general, el método de backtracking supera al método de tamaño de paso fijo en términos de precisión y generalización.

3. Conclusiones

A partir de los resultados obtenidos, se pueden extraer las siguientes conclusiones:

- El método de **backtracking** demuestra ser un algoritmo altamente preciso, logrando un error de entrenamiento de 0.0 y errores de validación y prueba significativamente bajos (0.00097 y 0.00047, respectivamente). Esto indica que el modelo ajusta perfectamente los datos de entrenamiento y generaliza muy bien a nuevos datos.

- Sin embargo, el método de backtracking tiene un costo computacional más alto en comparación con el método de tamaño de paso fijo, lo que lo hace más lento en términos de tiempo de ejecución. Esta lentitud se debe a la necesidad de realizar múltiples evaluaciones de la función objetivo y el gradiente durante el proceso de búsqueda del tamaño de paso óptimo en cada iteración.
- Por otro lado, el método de **tamaño de paso fijo** es más rápido computacionalmente, ya que no requiere cálculos adicionales para ajustar el tamaño de paso en cada iteración. Aunque su precisión es ligeramente inferior a la del backtracking, sigue siendo un método efectivo, con errores bajos en entrenamiento, validación y prueba (0.0054, 0.0054 y 0.0028, respectivamente).
- En resumen, la elección entre ambos métodos depende del contexto y los requisitos del problema. Si la precisión es la prioridad y se dispone de recursos computacionales suficientes, el método de backtracking es la mejor opción. Por el contrario, si se busca un equilibrio entre velocidad y precisión, el método de tamaño de paso fijo es una alternativa adecuada.